

COOKBOOK

Redis × Node.js Cookbook

Fifteen data structures, fifteen runnable programs

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

What this is

Fifteen runnable Node.js programs, one per Redis data structure, each built around a use case from the ICICI platform. Every program is standalone: read the header comment, run it, then read the code. The output narrates itself, so the terminal and the source tell the same story.

The code lives in `node_cookbook/` beside this PDF. What is printed here is the core excerpt from each file - the runnable version carries the narration and the cleanup as well.

	Detail
Recipes	15, one per data structure
Client	node-redis v6 (npm i redis) - also runs on v4.6+ and v5
Runtime	Node 18 or newer, ES modules
Server	Redis 7.x on 127.0.0.1:6379, or set REDIS_URL
Run all	<code>cd node_cookbook && npm install && npm run all</code>
Run one	<code>node src/07_zset_branch_leaderboard.js</code>

Setup, once

```
$ cd node_cookbook
$ npm install

# any Redis 7 will do
$ docker run -d --name redis-m07 -p 6379:6379 redis:7-alpine

$ npm run all           # all fifteen, in order
$ npm run r07          # just the leaderboard
$ REDIS_URL=redis://10.2.4.9:6379 npm run all
```

TIP Every recipe deletes only the keys it owns before it starts, so reruns are safe and nothing leaks between recipes. Nothing in this cookbook calls FLUSHALL.

The map

Find the row that matches what you are building, then open that recipe.

#	Structure	Use case	Key commands
01	String	Website hit counter	INCR · INCRBY · MGET · EXPIRE
02	String + TTL	Balance cache (cache-aside)	GET · SET · UNLINK · INFO
03	Hash	Customer profile cache	HSET · HGET · HINCRBY · OBJECT
04	List, capped	Last ten transactions	LPUSH · LTRIM · LRANGE · INDEX
05	List, two of them	Reliable statement queue	BLMOVE · LREM · LMOVE · LLEN
06	Set	Duplicate payment guard, segments	SADD · SISMEMBER · SINTER · SRANDMEMBER
07	Sorted Set	Live branch leaderboard	ZINCRBY · ZRANGE · ZREVRANK · ZREMRANGEBYRANK
08	String / Sorted Set	API rate limiting	INCR · ZREMRANGEBYSCORE · ZADD · MULTI
09	HyperLogLog	Unique visitors	PFADD · PFCOUNT · PFMERGE
10	Bitmap	Login streaks, daily active users	SETBIT · BITCOUNT · BITPOS · BITOP
11	Geospatial	Nearest ATM locator	GEOADD · GEOSEARCH · GEOSEARCH · GEODIST
12	Stream	Transaction events	XADD · XGROUP · XREADGROUP · XACK
13	Pub/Sub	Fraud alert fan-out	SUBSCRIBE · PSUBSCRIBE · PUBLISH · PUBSUB
14	String + Lua	Distributed lock, EOD batch	SET · EVAL · EVAL · PTTL
15	Transactions	Fund transfer, three ways	MULTI · WATCH · UNWATCH · EVAL

Choosing, from the question you are asking

If the question is...	Reach for	Recipe
"how many" of something that only goes up	String counter	01
"what was the answer last time"	String with a TTL	02
"this record, but one field at a time"	Hash	03
"the newest N"	List + LTRIM	04
"work that must be done exactly once"	List, or a Stream	05, 12
"have I seen this before"	Set	06
"top N", "rank of", "between X and Y"	Sorted Set	07
"how many times in the last minute"	Counter or Sorted Set	08
"how many distinct, roughly"	HyperLogLog	09
"on which days did this happen"	Bitmap	10
"what is near me"	Geospatial	11
"every consumer must see every event"	Stream	12
"tell everyone who is listening, now"	Pub/Sub	13
"only one server may do this"	String NX + Lua	14
"these two writes must hold together"	MULTI / WATCH / Lua	15

RECIPE 01 · STRING

Website hit counter

01_string_hit_counter.js

Counting page views on icicibank.com, per page and per day. The counter is touched by forty web servers at once, which is exactly the case a relational UPDATE handles badly and Redis handles by construction.

WHY THIS STRUCTURE A SQL UPDATE ... SET hits = hits + 1 takes a row lock and every other server queues behind it. INCR is atomic without any lock, because Redis executes one command at a time.

The commands that matter

Command	What it does for you
INCR key	Add 1, return the new value. A missing key starts at 0.
INCRBY key n	Add n - for batch imports, never GET-add-SET.
MGET k1 k2 k3	Read the whole dashboard in one round trip.
EXPIRE key secs	Daily counters delete themselves; no cron job.

The code

```
const pageKey = (page, day = today()) => `web:hits:${page}:${day}`;

async function recordHit(client, page) {
  const key = pageKey(page);

  // One round trip for both counters.
  const [pageHits] = await client
    .multi()
    .incr(key)
    .incr('web:hits:total')
    .expire(key, 60 * 60 * 24 * 40)
    .exec();

  return pageHits;
}

// 500 concurrent hits on one page
await Promise.all(
  Array.from({ length: 500 }, () => recordHit(client, 'accounts-savings')),
);
```

What it prints

```
GET web:hits:accounts-savings:2026-09-22
→ 501          exactly, not 487
OBJECT ENCODING web:hits:total
→ int          stored as a number, so INCR never parses
```

TIP A numeric string is encoded as int, which is why INCR stays O(1) however large the counter grows.

RECIPE 02 · STRING + TTL

Balance cache (cache-aside)

02_string_cache_aside.js

Balance enquiry is roughly 70% of retail API traffic and the balance changes rarely. Cache-aside is the pattern behind most of what Redis is used for, and the four rules in this recipe are the whole job.

WHY THIS STRUCTURE A 4 ms database round trip becomes a 0.1 ms memory read, and the core banking database stops being the thing that falls over at peak.

The commands that matter

Command	What it does for you
GET key	The hit path. Nothing else runs on a hit.
SET key v EX 300	Populate with a TTL - always, and jittered.
UNLINK key	Invalidate on write, on a background thread.
INFO stats	keyspace_hits / misses - is the cache earning its place?

The code

```

async function getBalance(client, acct) {
  const k = `icici:bal:${acct}`;

  try {
    // RULE 4 - fail open
    const cached = await client.get(k);
    if (cached !== null) return { balance: Number(cached), source: 'HIT' };
  } catch (err) { /* fall through to the database */ }

  const balance = await core.selectBalance(acct);
  if (balance === null) return { balance: null, source: 'MISS' };

  const jitter = Math.floor(Math.random() * 60); // RULE 1 - TTL + jitter
  await client.set(k, String(balance), { EX: 300 + jitter });
  return { balance, source: 'MISS' };
}

async function postDebit(client, acct, amount) {
  const balance = await core.applyDebit(acct, amount);
  await client.unlink(`icici:bal:${acct}`); // RULES 2 and 3
  return balance;
}

```

What it prints

```

MISS balance 341244.8 in 51.44 ms
HIT average 0.180 ms speed-up 286x
hit rate 99.3% below ~80% means the design is wrong

```

WATCH OUT Delete rather than overwrite. An overwrite can race with a read that is already on its way back with the old value, and writes it back after you.

RECIPE 03 · HASH

Customer profile cache

03_hash_customer_profile.js

The profile every screen in the mobile app needs. Fields are updated independently by different services, which is precisely what a JSON string cannot survive.

WHY THIS STRUCTURE A JSON string forces read-parse-modify-serialise-write for a one-field change, and two servers doing that at once lose an update. HINCRBY is one atomic server-side operation on one field.

The commands that matter

Command	What it does for you
HSET key f v [f v ...]	Write many fields; reply is the count of NEW ones.
HGET / HMGET	Read one field, or three - not the whole document.
HINCRBY key f n	Atomic counter inside the record.
OBJECT ENCODING	listpack below the threshold, hashtable above it.

The code

```
await client.hSet(`icici:cust:${id}`, {
  name: 'A. Sharma', tier: 'gold', logins: '41',
  branch: 'BR-MUM-014', mobile: '98xxxxxx21', kyc: 'verified',
});

// One field in, one field out - no client-side JSON at all
await client.hIncrBy(`icici:cust:${id}`, 'logins', 1); // → 42
await client.hGet(`icici:cust:${id}`, 'tier'); // → 'gold'
await client.hmGet(`icici:cust:${id}`, ['name', 'tier', 'branch']);

// HGETALL returns a plain JS object
const profile = await client.hGetAll(`icici:cust:${id}`);
```

What it prints

```
OBJECT ENCODING icici:cust:9003 → listpack    MEMORY USAGE → 176
... after adding 200 fields ...
OBJECT ENCODING icici:cust:9003 → hashtable  MEMORY USAGE → 15304
```

IMPORTANT Keeping each Hash under hash-max-listpack-entries (128) is the single biggest lever on Redis memory. The conversion is one-way - deleting fields does not convert it back.

RECIPE 04 · LIST, CAPPED

Last ten transactions

`04_list_recent_transactions.js`

The mini-statement on the app home screen. It never shows more than ten rows and it must render instantly, on every app open, for fifty million customers.

WHY THIS STRUCTURE `ORDER BY posted_at DESC LIMIT 10` hits the largest table in the bank every time the app opens. `LPUSH + LTRIM` keeps the answer ready: $O(1)$ to write, $O(10)$ to read, and bounded memory per account.

The commands that matter

Command	What it does for you
<code>LPUSH key v</code>	Push onto the head - newest first, for free.
<code>LTRIM key 0 9</code>	Discard everything past index 9. This is the cap.
<code>LRANGE key 0 9</code>	Read the window the screen needs.
<code>LINDEX key -1</code>	Negative indexes count back from the tail.

The code

```
const KEEP = 10;

async function recordTransaction(client, acct, txn) {
  await client
    .multi()
    .lPush(`icici:recent:${acct}`, JSON.stringify(txn))
    .lTrim(`icici:recent:${acct}`, 0, KEEP - 1)
    .exec();
}

// Render the mini-statement
const rows = await client.lRange(`icici:recent:${acct}`, 0, KEEP - 1);
rows.map((raw) => JSON.parse(raw));
```

What it prints

```
twelve transactions pushed, LLEN → 10
1. 09:12:04 txn:12 -1500
2. 09:11:04 txn:11 -1375
MEMORY USAGE icici:recent:9003 → 712 bytes for ten transactions
```

TIP The trim is what makes it a cap. Without it the list grows forever and you are back to writing a cleanup job.

RECIPE 05 · LIST, TWO OF THEM

Reliable statement queue

05_list_queue_worker.js

"Email me my statement" queues a PDF job. Several workers consume it, and a worker that crashes mid-job must not silently lose it.

WHY THIS STRUCTURE BRPOP alone deletes the job the instant the worker receives it - crash one line later and the work is gone with no trace. BLMOVE moves it somewhere you can find it again.

The commands that matter

Command	What it does for you
BLMOVE src dst RIGHT LEFT t	Claim a job atomically, blocking up to t seconds.
LREM list 1 job	Acknowledge: the job is finished.
LMOVE proc queue RIGHT LEFT	The janitor requeues what a dead worker left.
LLEN	Queue depth - the metric to alert on.

The code

```
const QUEUE = 'icici:q:statements';
const processingKey = (worker) => `${QUEUE}:processing:${worker}`;

// Claim - atomically moves the job to this worker's list
const raw = await client.blMove(QUEUE, processingKey(name), 'RIGHT', 'LEFT', 2);
if (raw === null) return; // timed out, queue is empty

await generatePdf(JSON.parse(raw));

// Acknowledge - only now does the job stop existing
await client.lRem(processingKey(name), 1, raw);

// The janitor, run on a timer
while (await client.lMove(processingKey(dead), QUEUE, 'RIGHT', 'LEFT')) {}
```

What it prints

```
w1 completed 9: stmt-10, stmt-9, stmt-8, ...
w2 picked stmt-4 and crashed before acknowledging it
LRANGE icici:q:statements:processing:w2 0 -1 → ["stmt-4"]
LMOVE ... → 1 job requeued w3 completed 1: stmt-4
```

WATCH OUT This is the hand-rolled version, and it is worth building once to understand the problem. In production use a Stream with consumer groups (recipe 12) and let Redis keep the books.

RECIPE 06 · SET

Duplicate payment guard, segments

06_set_duplicate_payments.js

The UPI gateway retries on timeout, so the same payment request arrives twice and must be posted once. Separately, marketing wants "gold customers on mobile with no credit card".

WHY THIS STRUCTURE SADD's return value IS the answer to "have I seen this before?" - one atomic round trip, with no SELECT-then-INSERT race. And SINTER/SDIFF do set algebra on the server that would otherwise be three joins on the OLTP database.

The commands that matter

Command	What it does for you
SADD key member	1 = new, 0 = already there. That reply is the decision.
SISMEMBER key m	O(1) whether the set holds 5 or 5 million members.
SINTER / SDIFF / SUNION	Segment algebra, executed inside Redis.
SRANDMEMBER / SPOP	Sample without removing / with removing.

The code

```
async function claimRequest(client, requestId) {
  const added = await client.sAdd('icici:seen:req', requestId);
  return added === 1;          // true = first time seen, process it
}

await claimRequest(client, 'REQ-88213'); // → true   post the payment
await claimRequest(client, 'REQ-88213'); // → false  reject the retry
await client.expire('icici:seen:req', 86400);

// Gold AND mobile AND NOT credit card
await client.sInterStore('icici:seg:tmp', ['icici:seg:gold', 'icici:seg:mobile']);
const target = await client.sDiff(['icici:seg:tmp', 'icici:seg:creditcard']);
```

What it prints

```
SADD icici:seen:req REQ-88213 → 1    first arrival
SADD icici:seen:req REQ-88213 → 0    duplicate, reject
SINTER gold mobile           → ["9003","9004","9009"]
SDIFF tmp creditcard         → ["9003"]
```

IMPORTANT Never write SISMEMBER followed by SADD. The gap between the two commands is exactly where the second gateway retry gets in.

RECIPE 07 · SORTED SET

Live branch leaderboard

07_zset_branch_leaderboard.js

A wallboard in the regional office ranking 4,000 branches by month-to-date volume, updated on every single payment.

WHY THIS STRUCTURE "Top 10 of 4,000" is $O(\log n + 10)$ and needs no ORDER BY over the transaction table. The ranking can never be stale, because the write that changes it is the write that maintains it.

The commands that matter

Command	What it does for you
ZINCRBY key n member	Read-modify-write on the server; no client ever reads.
ZRANGE k 0 2 REV WITHSCORES	Top three with scores, already sorted.
ZREVRANK key member	"Where am I?" in $O(\log n)$, 0-based.
ZREMRANGEBYRANK k 0 -101	Keep the top 100, drop the rest. Memory capped.

The code

```
const BOARD = `icici:lb:branch:${month}`;

// Called from the payment path
await client.zIncrBy(BOARD, amount, branch);

// The wallboard
const top = await client.zRangeWithScores(BOARD, 0, 2, { REV: true });
// → [{ value: 'BR-DEL-002', score: 410000 }, ...]

// "Where is my branch?"
const rank = await client.zRevRank(BOARD, 'BR-CHN-009'); // → 2
const score = await client.zScore(BOARD, 'BR-CHN-009'); // → 322000

// Nightly: keep only the top 100
await client.zRemRangeByRank(BOARD, 0, -101);
```

What it prints

```
1. BR-DEL-002   ■4,10,000
2. BR-MUM-014   ■3,40,000
3. BR-CHN-009   ■3,22,000
BR-CHN-009 is #3 of 5
```

TIP Use a timestamp as the score and the same structure becomes a time index: ZRANGEBYSCORE key <09:00> <10:00> is "everything in that hour".

RECIPE 08 · STRING / SORTED SET

API rate limiting

08_zset_rate_limiter.js

The public API allows each customer 5 calls a minute, and twenty API servers have to enforce that limit together. Counting in each Node process gives every server its own allowance and twenty times the real limit.

WHY THIS STRUCTURE Two implementations with an honest trade-off: a fixed window costs 8 bytes and one INCR but has a boundary burst; a sliding window is exact and costs ~70 bytes per request in flight.

The commands that matter

Command	What it does for you
INCR + EXPIRE NX	Fixed window. The NX stops a later call extending it.
ZREMRANGEBYSCORE	Sliding window: forget what fell out of it.
ZADD / ZCARD	One member per request, scored by timestamp.
MULTI	Both variants are still one round trip.

The code

```

async function allowFixed(client, customer) {
  const bucket = Math.floor(Date.now() / 1000 / WINDOW);
  const key = `icici:rl:fixed:${customer}:${bucket}`;
  const [used] = await client.multi()
    .incr(key).expire(key, WINDOW, 'NX').exec();
  return { allowed: used <= LIMIT, remaining: Math.max(0, LIMIT - used) };
}

async function allowSliding(client, customer) {
  const key = `icici:rl:slide:${customer}`;
  const now = Date.now();
  const replies = await client.multi()
    .zRemRangeByScore(key, 0, now - WINDOW * 1000)
    .zAdd(key, { score: now, value: `${now}-${rand()}` })
    .zCard(key)
    .expire(key, WINDOW + 1)
    .exec();
  return { allowed: Number(replies[2]) <= LIMIT };
}

```

What it prints

```

call 5  ALLOW  used 5  remaining 0
call 6  DENY   used 6  remaining 0  → HTTP 429, Retry-After

```

WATCH OUT The fixed window's cliff is real: five calls at 09:00:59 and five at 09:01:00 are ten in one second and both windows say "within limit". Use the sliding version on login, OTP and payment endpoints.

RECIPE 09 · HYPERLOGLOG

Unique visitors

`09_hll_unique_visitors.js`

"How many distinct customers used the app today? This week?" The number goes on a dashboard, and nobody needs it to be exact.

WHY THIS STRUCTURE Counting distinct things exactly means remembering every id you have seen. A HyperLogLog remembers none of them, is within about 0.81%, and costs 12 KB whether you feed it a thousand ids or a billion.

The commands that matter

Command	What it does for you
PFADD key element	Add an id. Nothing is stored - only registers are updated.
PFCOUNT key [key ...]	The estimate; over several keys it unions them first.
PFMERGE dst src [src ...]	Roll days up into a week without re-scanning.

The code

```
// One call per event, from the login path
await client.pfAdd(`icici:uv:hll:${day}`, `cust:${id}`);

const today = await client.pfCount(`icici:uv:hll:${day}`);

// Weekly uniques - you cannot just add the daily numbers,
// that double-counts everyone who came back
await client.pfMerge('icici:uv:hll:week-38', sevenDailyKeys);
const week = await client.pfCount('icici:uv:hll:week-38');
```

What it prints

```
PFCOUNT → 100156      SCARD → 100000      error 0.156%
MEMORY USAGE hll →    12,352 bytes    (fixed, forever)
MEMORY USAGE set →    6,291,560 bytes  (grows with every id)
```

IMPORTANT You can never ask "is customer 9003 in there?" - the ids are not stored. If you need that question answered, you need a Set and you will pay for it.

RECIPE 10 · BITMAP

Login streaks, daily active users

10_bitmap_login_streak.js

"Which customers logged in on at least 20 days this year?" and "who used the app on all three days?" - retention questions, asked constantly, answered badly by a table of login rows.

WHY THIS STRUCTURE One bit per customer per day. A full year of login history is 46 bytes per customer, and the cohort questions become bit arithmetic run by the CPU's popcount instruction.

The commands that matter

Command	What it does for you
SETBIT key offset 1	Record the event. Offset = day of year, or customer id.
BITCOUNT key [s e BIT]	How many days active. BIT ranges need Redis 7.
BITPOS key 1	First day they ever logged in.
BITOP AND dst k1 k2 k3	Cohorts: AND = retained, OR = MAU, XOR = churn.

The code

```
// Layout (a): one key per customer, one bit per day
await client.setBit(`icici:login:${acct}:2026`, dayOfYear(date), 1);

await client.bitCount(`icici:login:${acct}:2026`);           // days active
await client.bitCount(key, { start: 244, end: 273, mode: 'BIT' }); // September
await client.bitPos(`icici:login:${acct}:2026`, 1);          // first login

// Layout (b): one key per day, one bit per customer
await client.bitOp('AND', 'icici:dau:app:sticky',
  [...09-20', '...09-21', '...09-22']);
await client.bitCount('icici:dau:app:sticky'); // active all three days
```

What it prints

```
BITCOUNT icici:login:9003:2026    → 61 days active
MEMORY USAGE icici:login:9003:2026 → 88 bytes for the whole year
BITOP AND + BITCOUNT              → 1,666 customers active all 3 days
```

TIP Which layout you choose decides which questions are cheap. Key-per-customer answers "this customer's history"; key-per-day answers "this day's cohort". Many systems keep both.

RECIPE 11 · GEOSPATIAL

Nearest ATM locator

`11_geo_atm_locator.js`

"Show me ICICI ATMs within 3 km, nearest first" - the map screen in the mobile app, over an estate of twenty thousand ATMs.

WHY THIS STRUCTURE A radius query answered in microseconds without PostGIS, and without pulling the table into Node to compute haversine distances by hand.

The commands that matter

Command	What it does for you
GEOADD key lon lat member	Longitude FIRST. This is the usual bug.
GEOSEARCH ... BYRADIUS	Circle search, sorted by real distance.
GEOSEARCH ... BYBOX	Rectangle - the right shape for a map viewport.
GEODIST / GEOPOS	Distance between two members; coordinates of one.

The code

```
await client.geoAdd('icici:geo:atm:mumbai', [
  { longitude: 72.8347, latitude: 18.9220, member: 'ATM-COLABA-01' },
  { longitude: 72.8478, latitude: 19.0176, member: 'ATM-DADAR-11' },
]);

const near = await client.geoSearchWith(
  'icici:geo:atm:mumbai',
  { longitude: 72.8258, latitude: 18.9260 }, // where the customer is
  { radius: 5, unit: 'km' },
  ['WITHDIST', 'WITHCOORD'],
  { SORT: 'ASC', COUNT: 3 },
);
```

What it prints

```
1. ATM-CHURCHGATE-04    1.92 km
2. ATM-COLABA-01        1.98 km
3. ATM-MAHALAXMI-02     5.46 km
ZSCORE icici:geo:atm:mumbai ATM-DADAR-11 → 3331906285478848
```

TIP A geo index IS a Sorted Set whose score is a 52-bit geohash. ZREM removes an ATM, ZCARD counts them - there is no separate type to learn.

RECIPE 12 · STREAM

Transaction events

12_stream_txn_events.js

Every card transaction is published once and consumed by three independent teams - fraud scoring, the ledger feed, notifications. Nobody may miss an event, and a consumer that dies must not take its backlog with it.

WHY THIS STRUCTURE This is what Pub/Sub cannot do. A Stream keeps the message, tracks who has read it, and lets another consumer claim the work of one that died. It is Kafka's model, in Redis - and the bridge into Module 08.

The commands that matter

Command	What it does for you
XADD key * field value	Append. The id is <ms>-<seq>, so it is time-ordered.
XGROUP CREATE k g 0	A consumer group; each group tracks its own position.
XREADGROUP ... >	Deliver entries never yet given to this group.
XACK / XPENDING / XAUTOCLAIM	Confirm done · what is outstanding · take over.

The code

```
await client.xAdd(STREAM, '*', {
  txn: 'txn:1001', acct: '9003', amount: '1500', mcc: '5411',
}, { TRIM: { strategy: 'MAXLEN', strategyModifier: '~', threshold: 100000 } });

await client.xGroupCreate(STREAM, 'fraud-scoring', '0', { MKSTREAM: true });

const reply = await client.xReadGroup(
  'fraud-scoring', 'scorer-1', [{ key: STREAM, id: '>' }], { COUNT: 3 });

for (const m of reply[0].messages) {
  await score(m.message);
  await client.xAck(STREAM, 'fraud-scoring', m.id); // only now is it done
}

// Take over a dead consumer's backlog
const claimed = await client.xAutoClaim(STREAM, 'fraud-scoring', 'scorer-3', 0, '0');
```

What it prints

```
scorer-1 received 3: txn:1001, txn:1002, txn:1003   acknowledged all 3
scorer-2 received 3   crashed before acknowledging anything
XPENDING → 3 pending
XAUTOCLAIM → scorer-3 claimed 3   XPENDING → 0 pending
```

CHECKPOINT Nothing lost, nothing processed twice. That is the whole argument for Streams over Lists and over Pub/Sub.

RECIPE 13 · PUB/SUB

Fraud alert fan-out

13_pubsub_fraud_alerts.js

A fraud rule fires and every console open in the office must light up now. Also "I just evicted this key, drop your local copy" across twenty app servers.

WHY THIS STRUCTURE Fan-out to N listeners in one command with no broker to run - as long as losing a message while a subscriber reconnects is genuinely harmless.

The commands that matter

Command	What it does for you
SUBSCRIBE channel	On its own connection - a subscriber cannot run commands.
PSUBSCRIBE icici:alerts:*	One supervisor watching every alert channel.
PUBLISH channel message	Returns how many clients received it.
PUBSUB CHANNELS / NUMSUB	Who is listening right now.

The code

```
// A connection in subscriber mode cannot run normal commands,
// so the subscriber gets its own
const subscriber = client.duplicate();
await subscriber.connect();

await subscriber.subscribe('icici:alerts:fraud', (message) => {
  renderAlert(JSON.parse(message));
});

await supervisor.pSubscribe('icici:alerts:*', (message, channel) => {
  log(channel, message);
});

const delivered = await client.publish('icici:alerts:fraud',
  JSON.stringify({ acct: 9003, rule: 'CARD_NOT_PRESENT_VELOCITY' }));
```

What it prints

```
PUBLISH icici:alerts:fraud {...} → 3 clients received it
... console-2 disconnects ...
PUBLISH icici:alerts:fraud {...} → 2 clients received it
the alert console-2 missed is gone - there is no backlog
```

IMPORTANT PUBLISH returns the number of clients that got it. If that number being 0 would be a problem, you needed a Stream, not a channel.

RECIPE 14 · STRING + LUA

Distributed lock, EOD batch

14_lock_eod_batch.js

Reconciliation runs at 23:30 on six app servers. Exactly one must run it; twice would double-post every entry.

WHY THIS STRUCTURE Three parts, each protecting against a different failure: NX so only one caller wins, PX so a crashed holder does not block the batch forever, and a random token so a stalled process cannot release someone else's lock.

The commands that matter

Command	What it does for you
SET k token NX PX 30000	Acquire. The reply is (nil) for everyone who lost.
EVAL <compare-and-delete>	Release, atomically, only if the token is ours.
EVAL <compare-and-pexpire>	Extend the lease from a watchdog timer.
PTTL key	How long the current holder has left.

The code

```
const RELEASE = `
  if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("DEL", KEYS[1])
  else
    return 0
  end`;

async function acquire(client, ttlMs = 30000) {
  const token = randomUUID();
  const ok = await client.set(LOCK, token, { NX: true, PX: ttlMs });
  return ok ? token : null; // null = someone else holds it
}

async function release(client, token) {
  return client.eval(RELEASE, { keys: [LOCK], arguments: [token] });
}
```

What it prints

```
app-01 HOLDS the lock, token 7f3c9a11... - running reconciliation
app-02 did not get the lock - backing off (9,847 ms left on it)
app-03 did not get the lock - backing off (9,846 ms left on it)
servers that ran the batch: 1
```

WATCH OUT Correct on a single instance. With replication a failover can lose the lock write and hand it to two servers - if that would be a financial incident, use Redlock or keep the guarantee in PostgreSQL.

RECIPE 15 · TRANSACTIONS

Fund transfer, three ways

15_transaction_fund_transfer.js

Move money between two wallets without ever letting a balance go negative - the exercise that shows exactly what a Redis transaction is and is not.

WHY THIS STRUCTURE MULTI guarantees the queued commands run consecutively with nothing interleaved. It does NOT roll back. Any condition that must hold has to live inside WATCH or inside Lua, never in application code between two calls.

The commands that matter

Command	What it does for you
MULTI ... EXEC	Queue, then run consecutively. One array of replies.
WATCH key	EXEC returns null if that key changed. Retry the whole thing.
UNWATCH	Abandon cleanly when the check fails.
EVAL	Check and both writes in one uninterruptible step.

The code

```
// WATCH needs its own connection - the watch is per-connection.
const iso = client.duplicate();
await iso.connect();
await iso.watch(A);

const balance = Number(await iso.get(A));
if (balance < amount) { await iso.unwatch(); return 'INSUFFICIENT'; }

const replies = await iso.multi().decrBy(A, amount).incrBy(B, amount).exec();
if (replies === null) return retry(); // A changed - nothing was applied

// The Lua version: no retry loop, one round trip
const TRANSFER = `
  local balance = tonumber(redis.call("GET", KEYS[1]) or "0")
  if balance < tonumber(ARGV[1]) then return {0, balance} end
  redis.call("DECRBY", KEYS[1], ARGV[1])
  redis.call("INCRBY", KEYS[2], ARGV[1])
  return {1, balance - tonumber(ARGV[1])}`;
```

What it prints

```
MULTI with a wrong-type command in the middle:
  A before 9000 → after 9001    the failing command undid nothing
WATCH under contention: result OK on attempt 2   A=3000 B=6000
EVAL transfer 99000 → ok=false balance=10000    refused, one round trip
```

IMPORTANT Redis transactions give isolation, not rollback. If you expected SQL semantics, this is the recipe that corrects it - and the reason the ledger stays in PostgreSQL.

House rules these recipes follow

Every recipe in this cookbook obeys the same six conventions. They are not style preferences - each one is there because its absence has caused a production incident somewhere.

Convention	Why it is there
icici:<thing>:<id> key names	The colon is the de facto namespace separator; RedisInsight groups by it and --scan --pattern relies on it
Every cache key gets a TTL	A key that nothing invalidates is wrong forever, and memory only goes one way
UNLINK, not DEL	Frees the memory on a background thread instead of on the one command thread
SCAN, never KEYS	KEYS is $O(n)$ over the whole keyspace and blocks every other client while it runs
MULTI for related writes	One round trip, and no other client can observe a half-applied state
Cache errors are swallowed	Redis becoming unreachable must degrade the API, not take it down

Where each recipe comes from

Recipe	Deck	Lab	Exercise
01, 02 · String	5	7.3, 7.9	5, 6
03 · Hash	3, 5	7.4	3
04, 05 · List	3, 5	7.5	2, 8
06 · Set	3, 5	7.5	2
07, 08 · Sorted Set	3, 5	7.6	2, 5
09, 10, 11 · HLL, Bitmap, Geo	3	7.6	2
12, 13 · Stream, Pub/Sub	5	—	9
14, 15 · Lock, transactions	5	7.10	7

HOW TO USE THIS The exercise document asks you to write several of these yourself before you read the version here. Do them in that order - typing it wrong once is worth more than reading it right twice.